Product Analyzer with Higher-Order Functions

```
// 🧩 Product Analyzer with Higher-Order Functions


// ------------------------------
// TODO 1: Product Class
// ------------------------------
data class Product(
    val name: String,
    val category: String,
    val price: Double,
    val stock: Int
)


// ------------------------------
// TODO 2: applyDiscount Function
// ------------------------------
fun applyDiscount(
    products: List<Product>,
    discountFunction: (Double) -> Double
): List<Product> {
    return products.map { product ->
        product.copy(price = discountFunction(product.price))
    }
}


// ------------------------------
// TODO 3: filterProducts Function
// ------------------------------
fun filterProducts(
    products: List<Product>,
    condition: (Product) -> Boolean
): List<Product> {
    return products.filter(condition)
```

```
}

// -------------------------------
// TODO 4 & 5: Main Implementation
// -------------------------------
fun main() {

    // Sample Product List
    val productList = listOf(
        Product("Laptop", "Electronics", 100000.0, 5),
        Product("Phone", "Electronics", 50000.0, 15),
        Product("Chair", "Furniture", 2000.0, 25),
        Product("Table", "Furniture", 3000.0, 5),
        Product("Pen", "Stationery", 50.0, 100)
    )

    // Apply 10% Discount
    val discountedProducts = applyDiscount(productList) { price ->
        price * 0.9
    }

    println("\n--- Discounted Product List ---")
    for (product in discountedProducts) {
        println(
            "Name: ${product.name} | " +
            "Category: ${product.category} | " +
            "Price: ${product.price} | " +
            "Stock: ${product.stock}"
        )
    }

    // Filter: Products with stock < 10
    val lowStockProducts = filterProducts(discountedProducts) {
```

```
        it.stock < 10
    }

    println("\n--- Low Stock Products (<10) ---")
    for (product in lowStockProducts) {
        println("${product.name} (${product.stock})")
    }

    // Filter: Products with price > 1000
    val premiumProducts = filterProducts(discountedProducts) {
        it.price > 1000
    }

    println("\n--- Premium Products (>1000) ---")
    for (product in premiumProducts) {
        println("${product.name} (${product.price})")
    }
}
```